

Problem 1 - Each Explanatory Variable

```
import numpy as np
import pandas as pd
from google.colab import drive
import matplotlib.pyplot as plt
from IPython.display import display

drive.mount('/content/drive')

file_path = '/content/drive/MyDrive/Machine Learning/D3.csv'
D3 = pd.DataFrame(pd.read_csv(file_path))

X1 = D3.values[:, 0]
X2 = D3.values[:, 1]
X3 = D3.values[:, 2]
Y = D3.values[:, 3]

numOfRows = X1.size
X0 = np.ones((numOfRows, 1))

X1 = X1.reshape(numOfRows, 1)
X1 = np.hstack((X0, X1))

X2 = X2.reshape(numOfRows, 1)
X2 = np.hstack((X0, X2))

X3 = X3.reshape(numOfRows, 1)
X3 = np.hstack((X0, X3))

def compute_cost(X, y, theta):

    predictions = X.dot(theta)
    errors = np.subtract(predictions, Y)
    sqrErrors = np.square(errors)
    J = 1 / (2 * numOfRows) * np.sum(sqrErrors)
    return J
```

```

def gradient_descent(X, Y, theta, alpha, iterations):

    m = len(Y)
    cost_history = np.zeros(iterations)

    for i in range(iterations):
        predictions = X.dot(theta)
        errors = np.subtract(predictions, Y)
        sum_delta = (alpha / m) * X.transpose().dot(errors);
        theta = theta - sum_delta;
        cost_history[i] = compute_cost(X, Y, theta)

    return theta, cost_history

theta_X01 = [0., 0.]
theta_X02 = [0., 0.]
theta_X03 = [0., 0.]
iterations = 1500
alpha_01 = 0.01

theta_X01, cost_history_X01 = gradient_descent(X1, Y, theta_X01, alpha_01, iterations)
theta_X02, cost_history_X02 = gradient_descent(X2, Y, theta_X02, alpha_01, iterations)
theta_X03, cost_history_X03 = gradient_descent(X3, Y, theta_X03, alpha_01, iterations)

theta_X1 = [0., 0.]
theta_X2 = [0., 0.]
theta_X3 = [0., 0.]
alpha_1 = 0.1

theta_X1, cost_history_X1 = gradient_descent(X1, Y, theta_X1, alpha_1, iterations)
theta_X2, cost_history_X2 = gradient_descent(X2, Y, theta_X2, alpha_1, iterations)
theta_X3, cost_history_X3 = gradient_descent(X3, Y, theta_X3, alpha_1, iterations)

print('Final value of theta X01 =', theta_X01)
print('Final value of theta X02 =', theta_X02)
print('Final value of theta X03 =', theta_X03)

print('cost_history X01 =', cost_history_X01)
print('cost_history X02 =', cost_history_X02)
print('cost_history X03 =', cost_history_X03)

plt.scatter(X1[:, 1], Y, color='red', label='Training Data')

```

```
plt.plot(X1[:, 1], X1.dot(theta_X01), color='green', label='Linear Regression')
plt.rcParams["figure.figsize"] = (10, 6)
plt.grid(True)
plt.xlabel('X1')
plt.ylabel('D3')
plt.title('Linear Regression Fit X01')
plt.legend()
plt.show()
```

```
plt.plot(range(1, iterations + 1), cost_history_X01, color='blue')
plt.rcParams["figure.figsize"] = (10, 6)
plt.grid(True)
plt.xlabel('Number of iterations')
plt.ylabel('Cost (J)')
plt.title('Convergence of gradient descent X01')
plt.show()
```

```
plt.scatter(X2[:, 1], Y, color='red', label='Training Data')
plt.plot(X2[:, 1], X2.dot(theta_X02), color='green', label='Linear Regression')
plt.rcParams["figure.figsize"] = (10, 6)
plt.grid(True)
plt.xlabel('X2')
plt.ylabel('D3')
plt.title('Linear Regression Fit X02')
plt.legend()
plt.show()
```

```
plt.plot(range(1, iterations + 1), cost_history_X02, color='blue')
plt.rcParams["figure.figsize"] = (10, 6)
plt.grid(True)
plt.xlabel('Number of iterations')
plt.ylabel('Cost (J)')
plt.title('Convergence of gradient descent X02')
plt.show()
```

```
plt.scatter(X3[:, 1], Y, color='red', label='Training Data')
plt.plot(X3[:, 1], X3.dot(theta_X03), color='green', label='Linear Regression')
plt.rcParams["figure.figsize"] = (10, 6)
plt.grid(True)
plt.xlabel('X3')
plt.ylabel('D3')
plt.title('Linear Regression Fit X03')
plt.legend()
```

```

plt.show()

plt.plot(range(1, iterations + 1), cost_history_X03, color='blue')
plt.rcParams["figure.figsize"] = (10, 6)
plt.grid(True)
plt.xlabel('Number of iterations')
plt.ylabel('Cost (J)')
plt.title('Convergence of gradient descent X03')
plt.show()

print('Final value of theta X1 =', theta_X1)
print('Final value of theta X2 =', theta_X2)
print('Final value of theta X3 =', theta_X3)

print('cost_history X1 =', cost_history_X1)
print('cost_history X2 =', cost_history_X2)
print('cost_history X3 =', cost_history_X3)

plt.scatter(X1[:, 1], Y, color='red', label='Training Data')
plt.plot(X1[:, 1], X1.dot(theta_X1), color='green', label='Linear Regression')
plt.rcParams["figure.figsize"] = (10, 6)
plt.grid(True)
plt.xlabel('X1')
plt.ylabel('D3')
plt.title('Linear Regression Fit X1')
plt.legend()
plt.show()

plt.plot(range(1, iterations + 1), cost_history_X1, color='blue')
plt.rcParams["figure.figsize"] = (10, 6)
plt.grid(True)
plt.xlabel('Number of iterations')
plt.ylabel('Cost (J)')
plt.title('Convergence of gradient descent X1')
plt.show()

plt.scatter(X2[:, 1], Y, color='red', label='Training Data')
plt.plot(X2[:, 1], X2.dot(theta_X2), color='green', label='Linear Regression')
plt.rcParams["figure.figsize"] = (10, 6)
plt.grid(True)
plt.xlabel('X2')
plt.ylabel('D3')

```

```
plt.title('Linear Regression Fit X2')
plt.legend()
plt.show()

plt.plot(range(1, iterations + 1), cost_history_X2, color='blue')
plt.rcParams["figure.figsize"] = (10, 6)
plt.grid(True)
plt.xlabel('Number of iterations')
plt.ylabel('Cost (J)')
plt.title('Convergence of gradient descent X2')
plt.show()

plt.scatter(X3[:, 1], Y, color='red', label='Training Data')
plt.plot(X3[:, 1], X3.dot(theta_X3), color='green', label='Linear Regression')
plt.rcParams["figure.figsize"] = (10, 6)
plt.grid(True)
plt.xlabel('X3')
plt.ylabel('D3')
plt.title('Linear Regression Fit X3')
plt.legend()
plt.show()

plt.plot(range(1, iterations + 1), cost_history_X3, color='blue')
plt.rcParams["figure.figsize"] = (10, 6)
plt.grid(True)
plt.xlabel('Number of iterations')
plt.ylabel('Cost (J)')
plt.title('Convergence of gradient descent X3')
plt.show()
```

Problem 1 - Results

First I tested the model with $\alpha_{01} = 0.01$, noted by the notation X01, ... X03, thetaX01, ... theta X03, and cost_historyX01, ... cost_historyX03, etc.

The results are the following:

Final value of theta X01 = [5.71850653 -1.9568206]

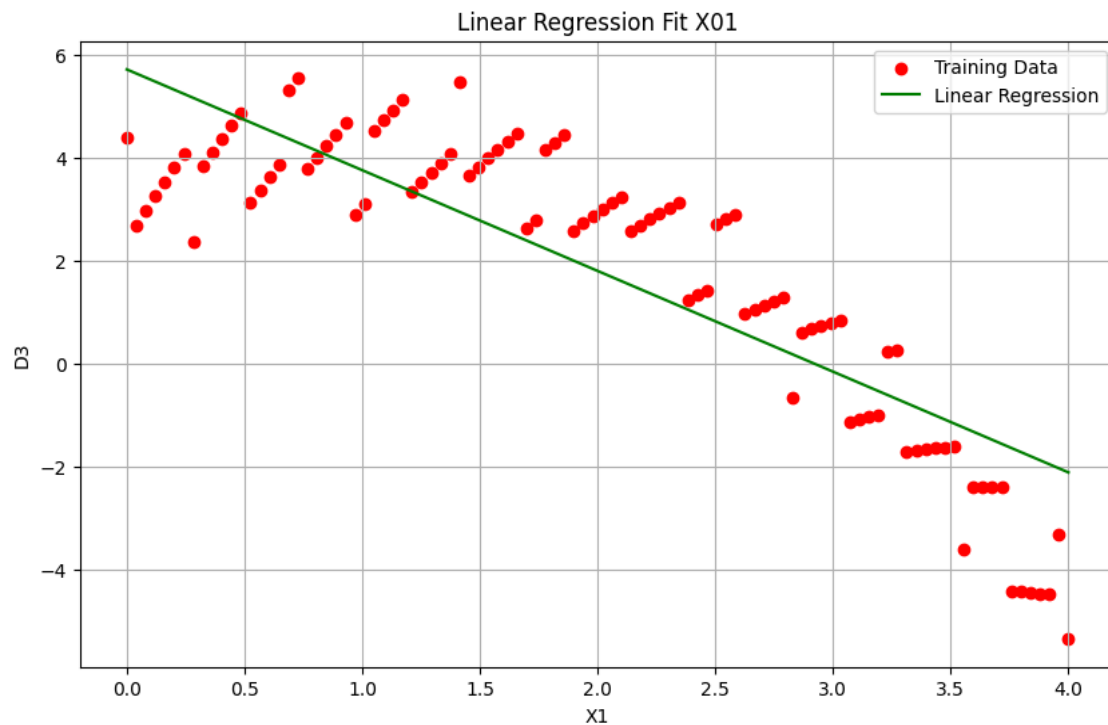
Final value of theta X02 = [0.71988473 0.56390334]

Final value of theta X03 = [2.78048129 -0.48451631]

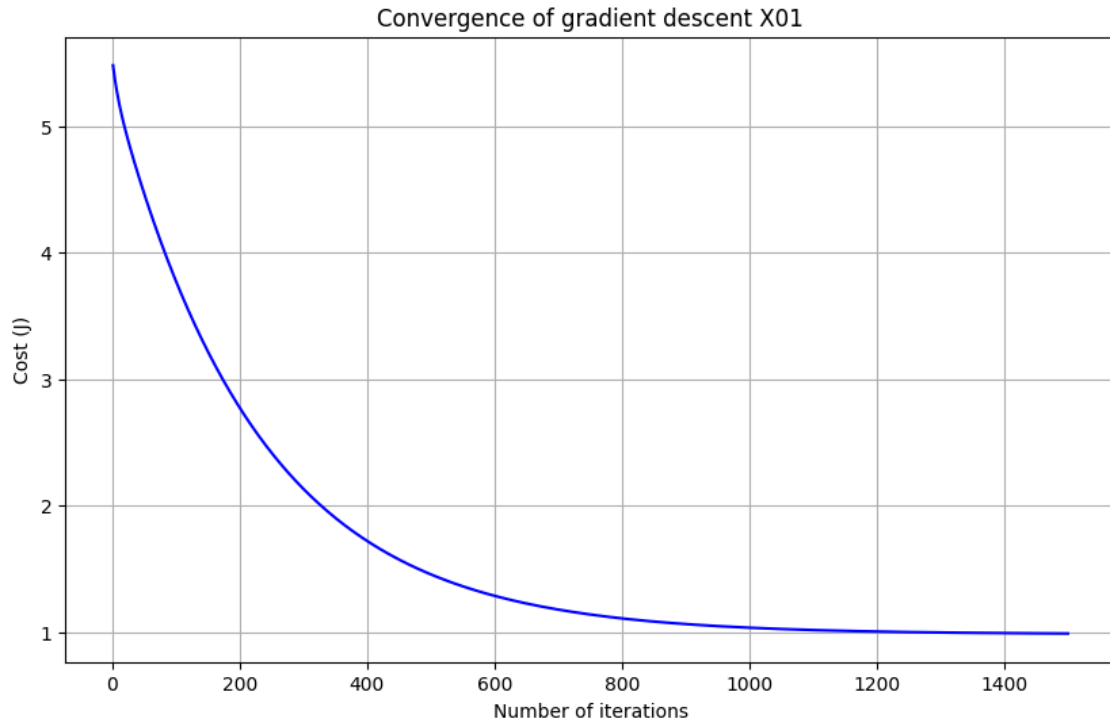
cost_history X01 = [5.48226715 5.44290965 5.40604087 ... 0.99063932 0.99061433 0.99058944]

cost_history X02 = [5.29831663 5.09909109 4.92356115 ... 3.5993997 3.59939955 3.5993994]

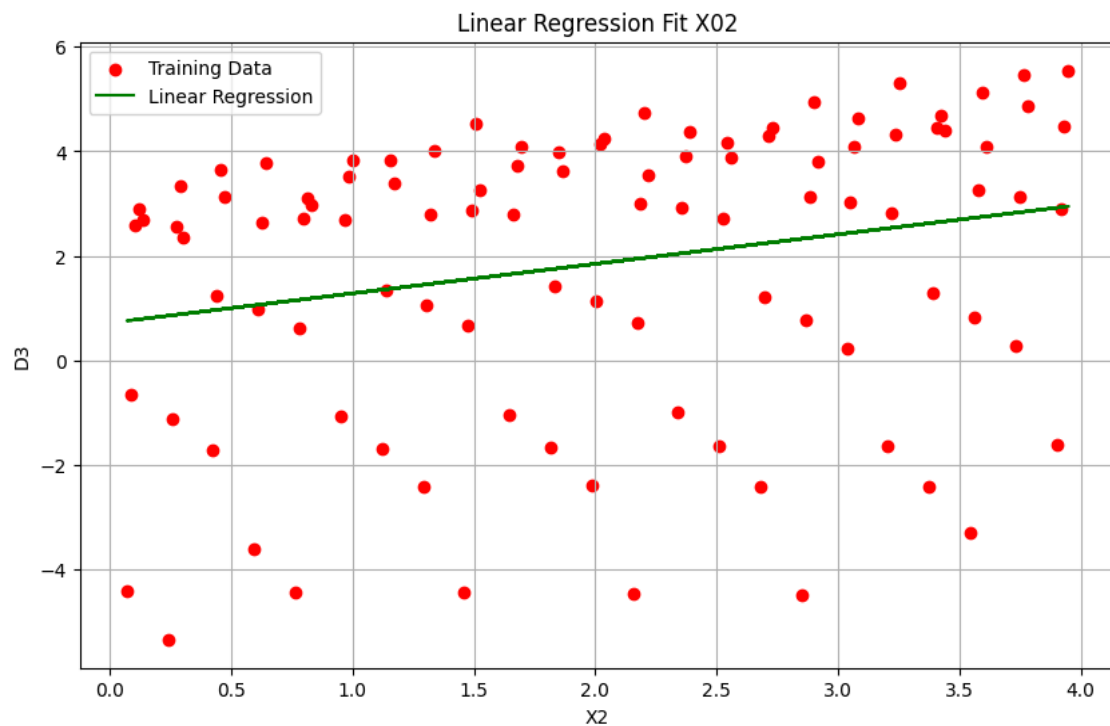
cost_history X03 = [5.40768785 5.30397076 5.21178297 ... 3.63053597 3.6305311 3.63052625]



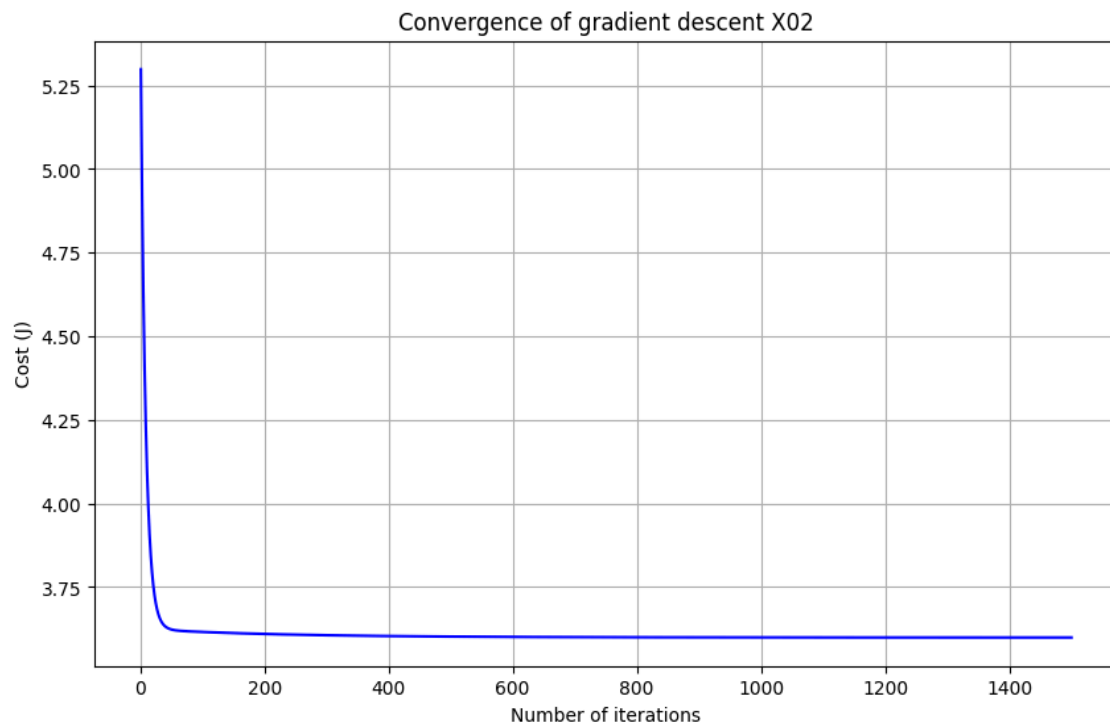
The fitted line shows a clear downward trend (theta = -1.96), consistent with X1 being the strongest predictor of Y.



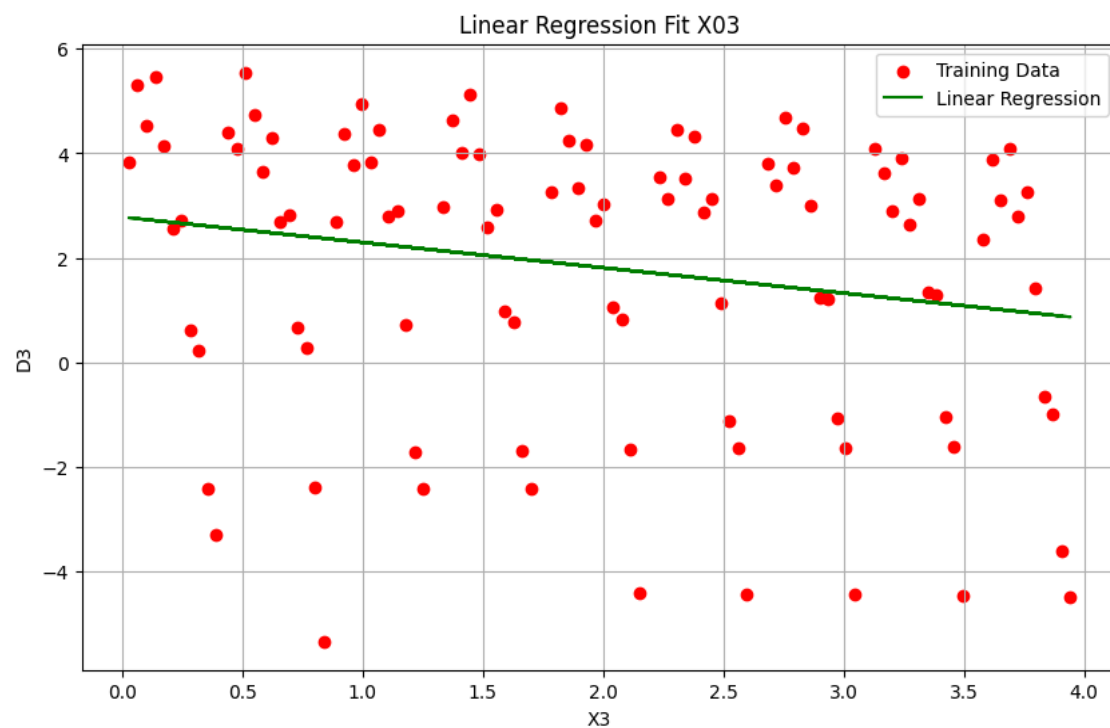
Cost decreases smoothly and has largely flattened by iteration 1500, though $X01$'s curve is still very slightly decreasing at the final iteration.



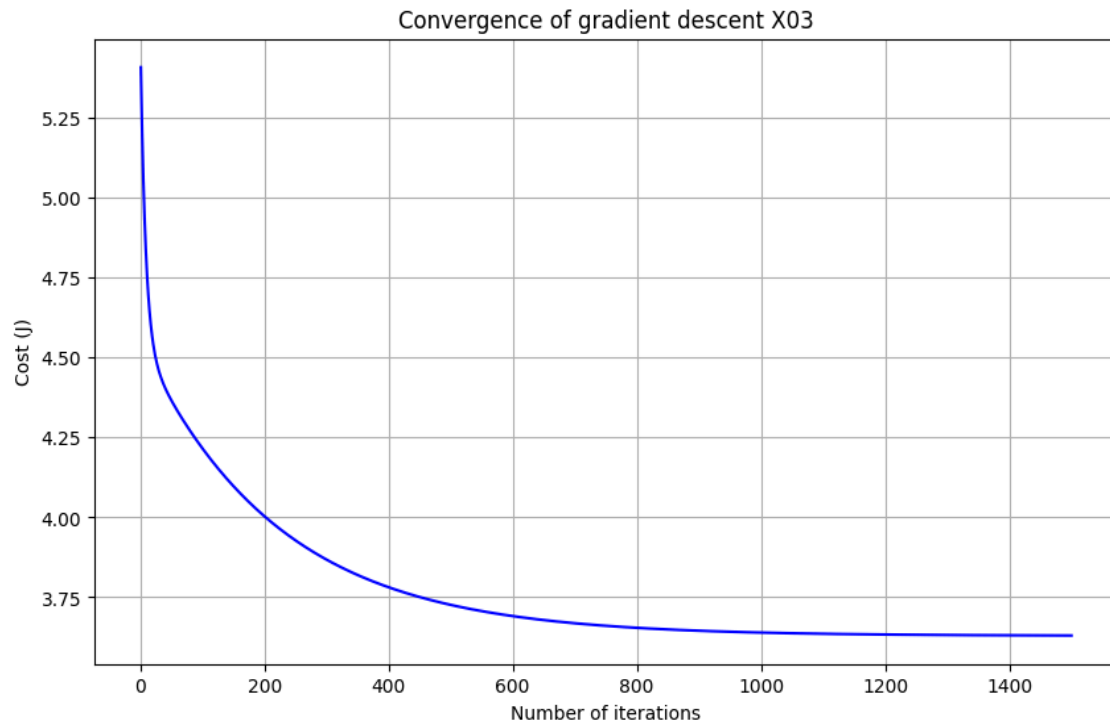
The fitted line shows a modest upward trend ($\theta = 0.564$), consistent with $X2$'s high final cost (~ 3.60) and weak explanatory power.



Cost decreases smoothly and has largely flattened by iteration 1500.



The fitted line shows a modest downward trend ($\theta = -0.484$), consistent with $X3$'s high final cost (~ 3.60) and weak explanatory power.



Cost decreases smoothly and has largely flattened by iteration 1500.

Afterwards, I tested the model with $\alpha_1 = 0.1$, noted by the notation X1, ... X3, thetaX1, ... theta X3, and cost_historyX1, ... cost_historyX3, ... etc.

The results are the following:

Final value of theta X1 = [5.92794892 -2.03833663]

Final value of theta X2 = [0.73606043 0.55760761]

Final value of theta X3 = [2.8714221 -0.52048288]

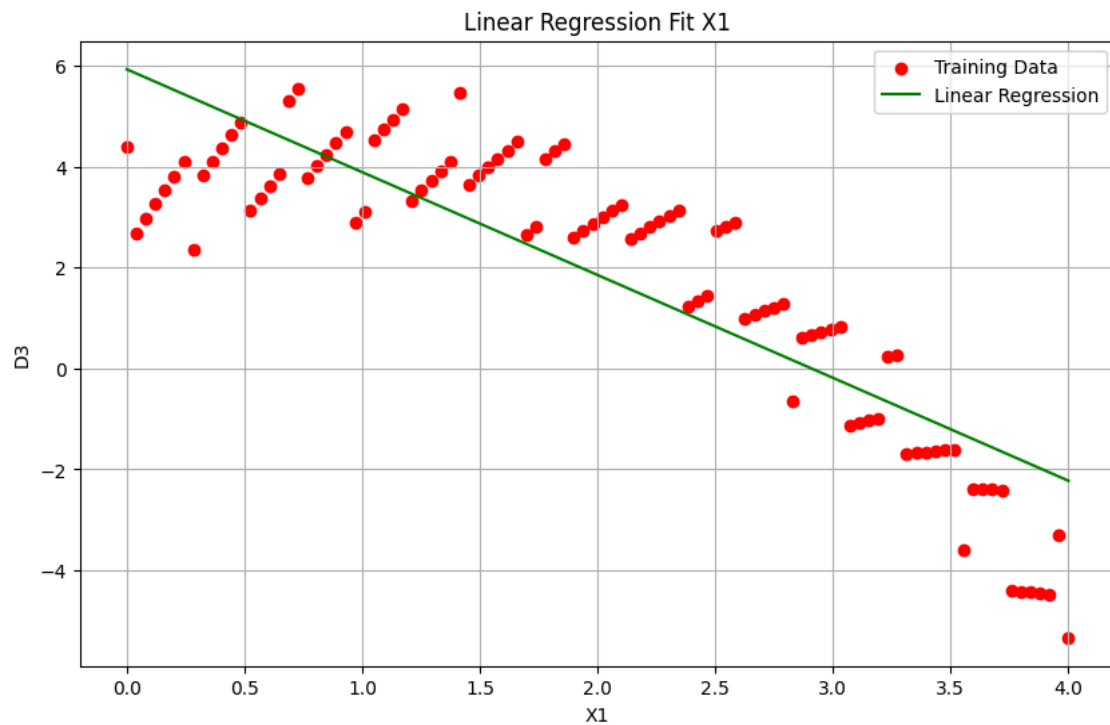
cost_history X1 = [5.16999006 4.96338989 4.7855721 ... 0.98499308 0.98499308 0.98499308]

cost_history X2 = [3.90731819 3.66528504 3.62832072 ... 3.59936602 3.59936602
3.59936602]

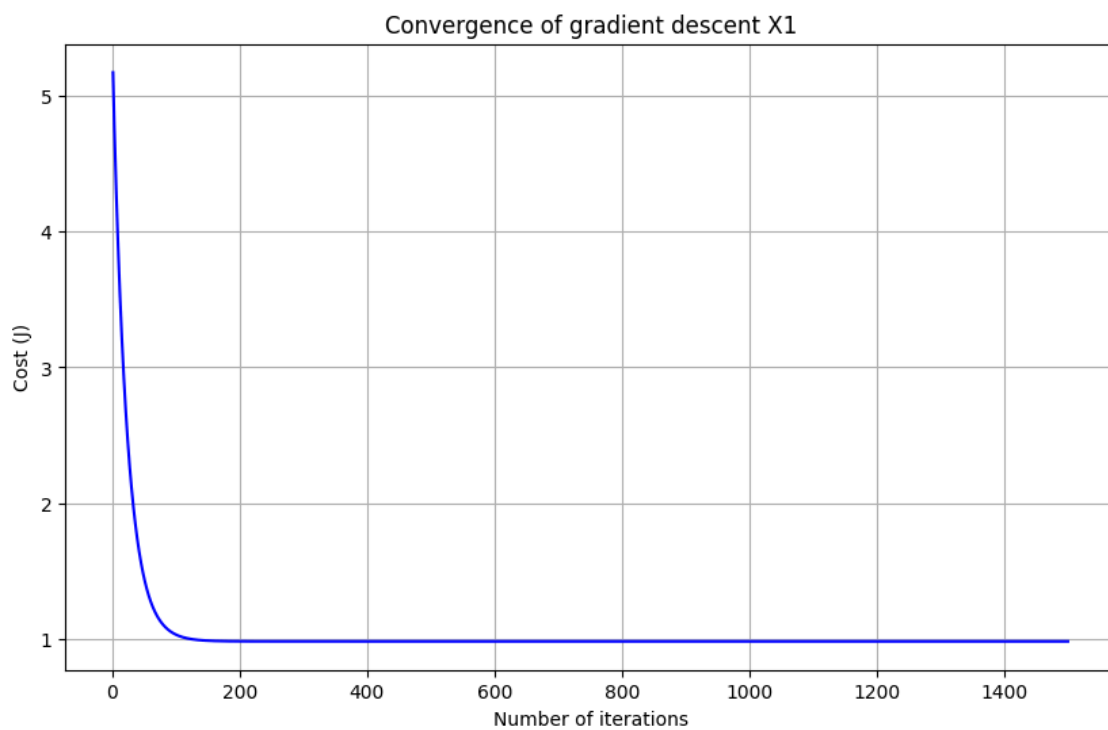
cost_history X3 = [4.66843939 4.49602325 4.43685075 ... 3.62945112 3.62945112 3.62945112]

The impacts of different alpha values on the model are discussed in a separate report titled "Machine Learning HW#1 Answers."

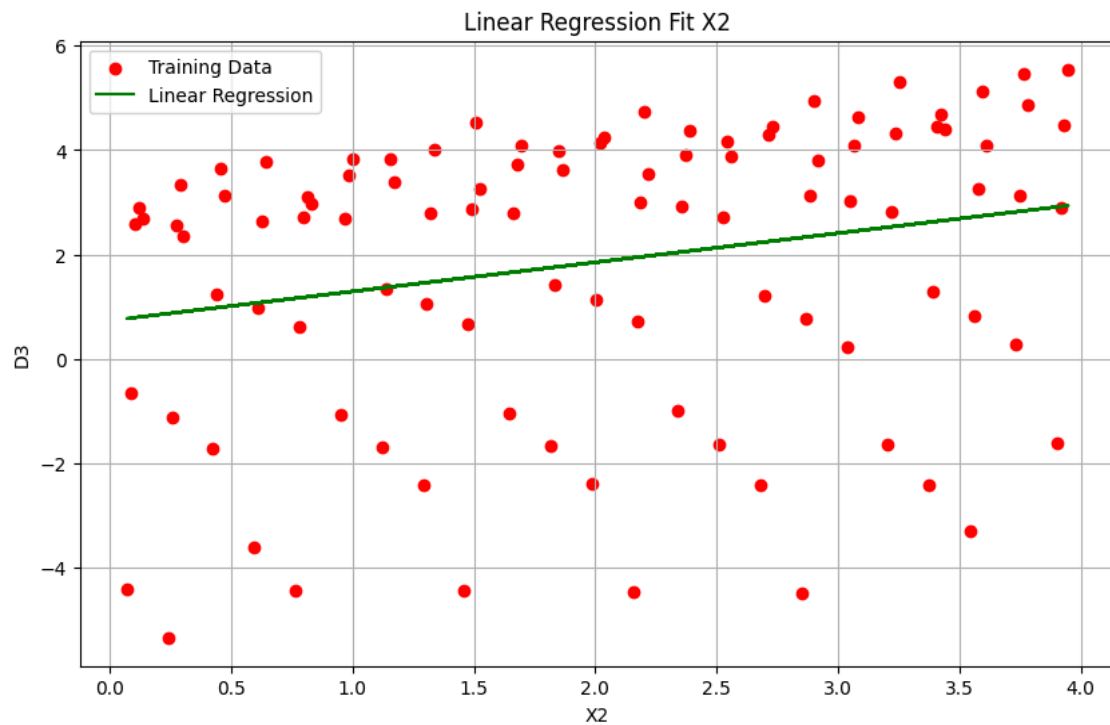
But in general, we can observe that the models provide the expected output. Take the plot "D3 vs Linear Regression Fit X01" for example, the green linear regression line follows a downward slope consistent with the decreasing trend in X1 values. This behavior can also be seen in X2 and X3 plots. Alpha value 0.1 proved to provide the more accurate theta values while providing the lowest loss over iteration as seen in the remaining plots below.



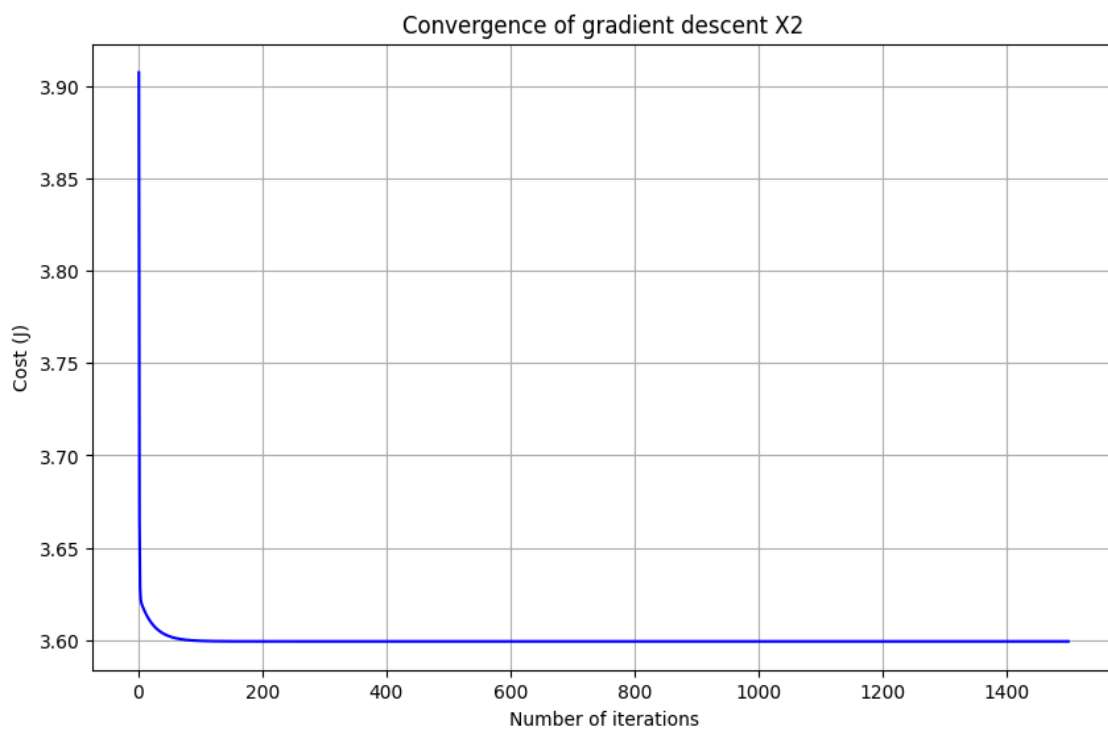
The fitted line shows a clear downward trend ($\theta = -2.04$), consistent with X1 being the strongest individual predictor of Y.



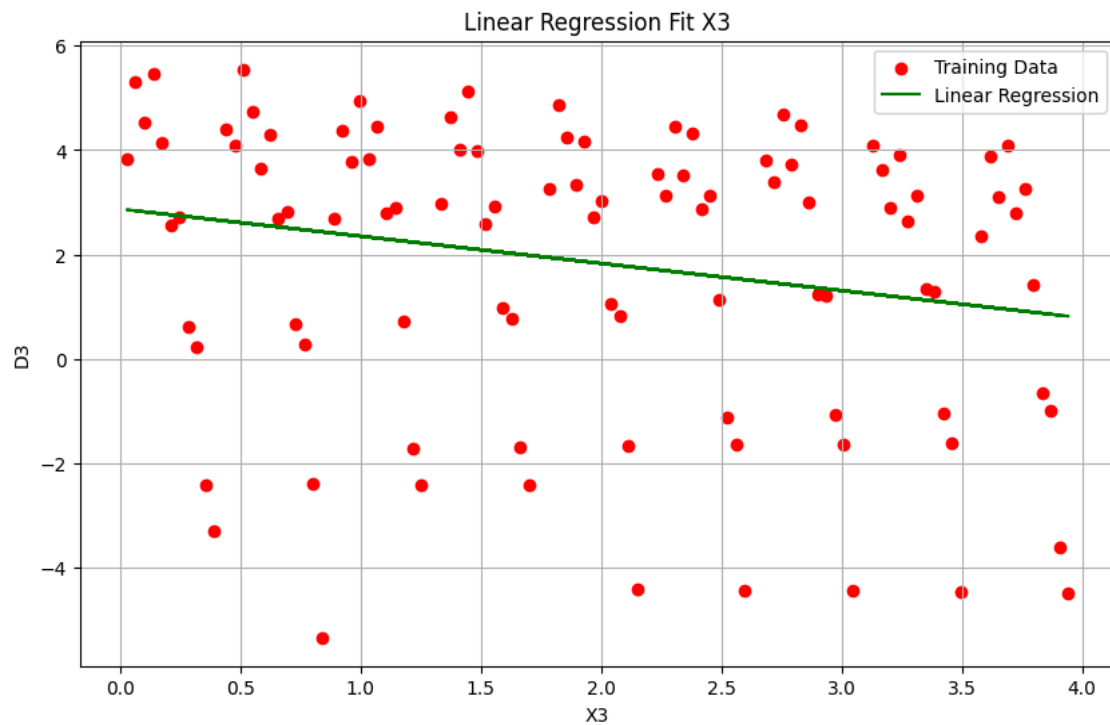
Cost converges rapidly, flattening well before iteration 1500.



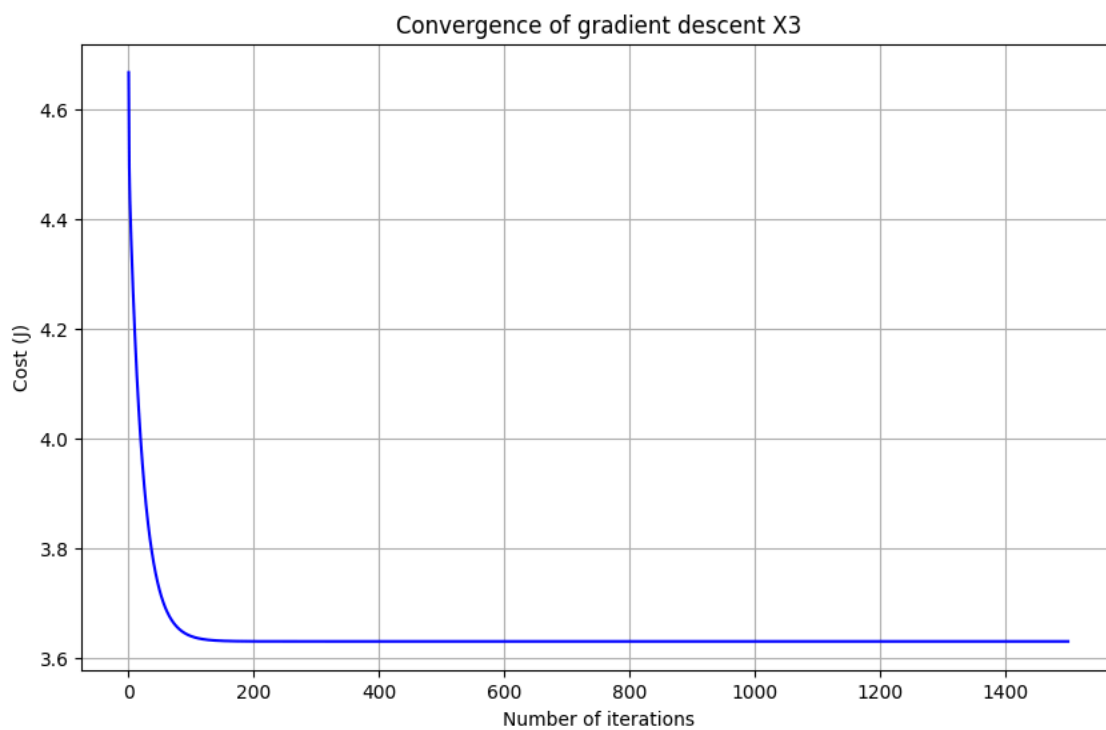
The fitted line shows a modest upward trend ($\theta = 0.56$) with substantial scatter, consistent with X2's high final cost (~ 3.60) and weak explanatory power.



Cost converges rapidly, flattening well before iteration 1500.



The fitted line shows a modest downward trend ($\theta = -0.52$), but the wide scatter reflects X3's limited ability to explain Y on its own.



Cost converges rapidly, flattening well before iteration 1500.

Problem 2

```
import numpy as np
import pandas as pd
from google.colab import drive
import matplotlib.pyplot as plt
from IPython.display import display

drive.mount('/content/drive')
file_path = '/content/drive/MyDrive/Machine Learning/D3.csv'
D3 = pd.DataFrame(pd.read_csv(file_path))

X1 = D3.values[:, 0]
X2 = D3.values[:, 1]
X3 = D3.values[:, 2]
Y = D3.values[:, 3]

numOfRows = X1.size
X0 = np.ones((numOfRows, 1))

def compute_cost(X, y, theta):

    predictions = X.dot(theta)
    errors = np.subtract(predictions, Y)
    sqrErrors = np.square(errors)
    J = 1 / (2 * numOfRows) * np.sum(sqrErrors)
    return J

def gradient_descent(X, Y, theta, alpha, iterations):

    m = len(Y)
    cost_history = np.zeros(iterations)

    for i in range(iterations):
        predictions = X.dot(theta)
        errors = np.subtract(predictions, Y)
        sum_delta = (alpha / m) * X.transpose().dot(errors);
        theta = theta - sum_delta;
        cost_history[i] = compute_cost(X, Y, theta)

    return theta, cost_history
X1 = X1.reshape(numOfRows,1)
X2 = X2.reshape(numOfRows,1)
```

```

X3 = X3.reshape(numOfRows,1)

X_all_vars = np.hstack((X0, X1, X2, X3))

theta_all_vars = np.zeros(4)

iterations = 1500
alpha_01 = 0.01
alpha_1 = 0.1
theta_all_vars_01, cost_history_all_vars_01 = gradient_descent(X_all_vars,
Y, theta_all_vars, alpha_01, iterations)

print('Final value of theta X_all_vars_01 =', theta_all_vars_01)

print('cost_history X_all_vars_01 =', cost_history_all_vars_01)

theta_all_vars_1, cost_history_all_vars_1 = gradient_descent(X_all_vars, Y,
theta_all_vars, alpha_1, iterations)

print('Final value of theta X_all_vars_1 =', theta_all_vars_1)

print('cost_history X_all_vars_1 =', cost_history_all_vars_1)

plt.plot(range(1, iterations + 1), cost_history_all_vars_1, color='blue')
plt.rcParams["figure.figsize"] = (10, 6)
plt.grid(True)
plt.xlabel('Number of iterations')
plt.ylabel('Cost (J)')
plt.title('Convergence of gradient descent X_all_vars_1')
plt.show()

```

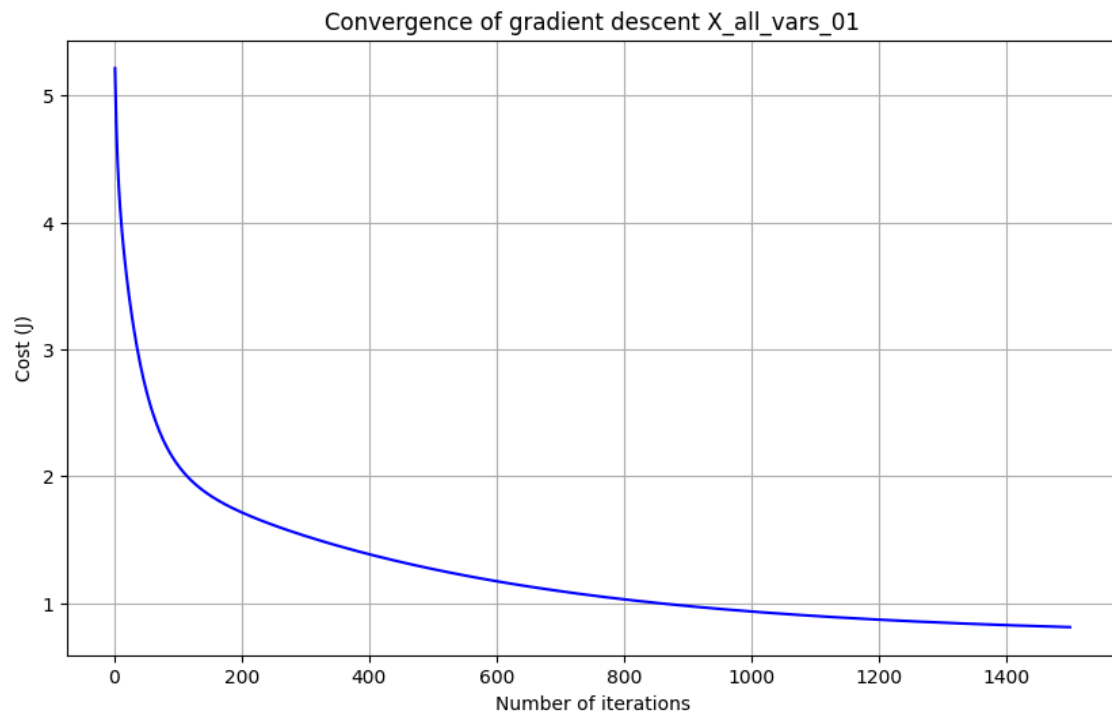
Final value of theta X_all_vars_01 = [4.15118728 -1.8394291 0.72473856 -0.09513266]

cost_history X_all_vars_01 = [5.21542243 4.97171977 4.7765543 ... 0.81079843 0.81065429
0.81051044]

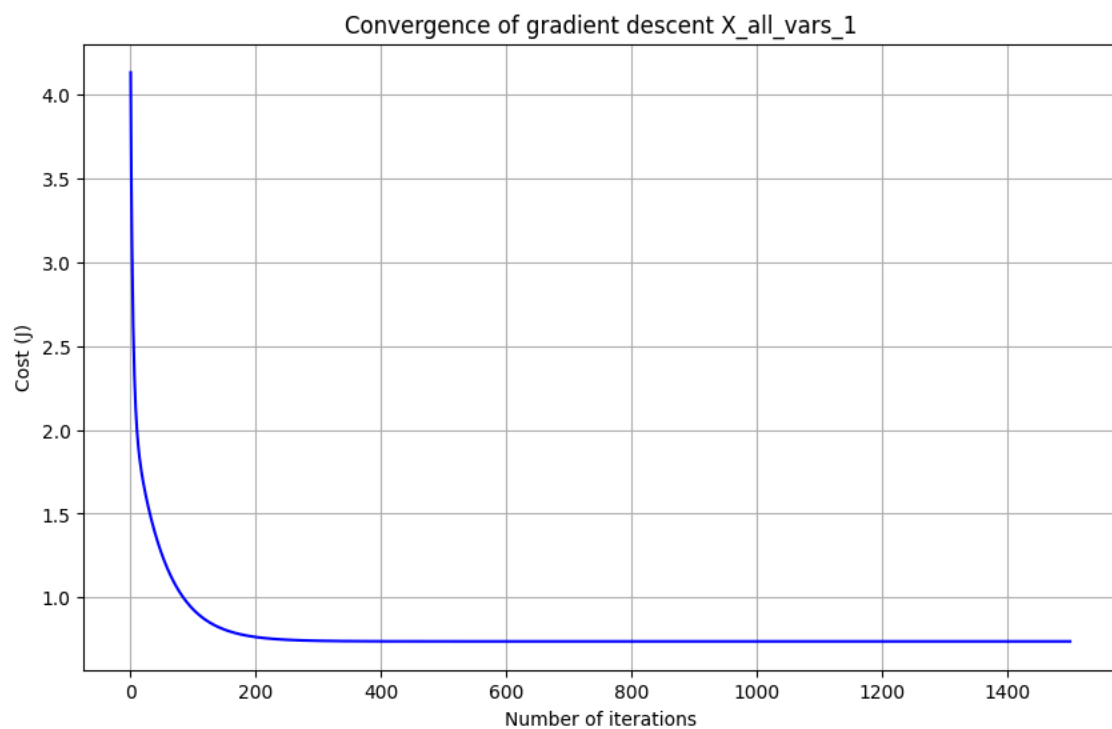
Final value of theta X_all_vars_1 = [5.31416563 -2.00371905 0.53256359 -0.26560164]

cost_history X_all_vars_1 = [4.13064348 3.51770697 3.12758306 ... 0.73846424 0.73846424
0.73846424]

As mentioned previously in problem 1, an alpha value of 0.1 proved to be most promising. Which is why I used it again here in problem 2, this time using all three explanatory variables together. This required a bigger matrix that holds X0, X1, X2, and X3.



Cost decreases steadily but has not fully flattened by iteration 1500, indicating the model had not yet converged.



Cost converges quickly, flattening well before iteration 1500.